P2P 기본 원리 및 libp2p 통합

개요

IPFS는 libp2p 라이브러리를 기반으로 한 분산 P2P(Peer-to-Peer) 네트워크를 구축합니다. 이 문서는 IPFS에서 libp2p가 어떻게 통합되고 활용되는지, 그리고 피어 간 통신의 기본 원리를 설명합니다.

libp2p 아키텍처 개요

libp2p는 모듈화된 네트워킹 스택으로, 다음과 같은 계층으로 구성됩니다:

IPFS에서의 libp2p 통합

1. 호스트 초기화

IPFS 노드는 `core/node/libp2p/host.go` 에서 libp2p 호스트를 초기화합니다:

```
func Host(mctx helpers.MetricsCtx, lc fx.Lifecycle, params P2PHostIn) (out P2PH

opts := []libp2p.Option{libp2p.NoListenAddrs}

for _, o := range params.Opts {

opts = append(opts, o...)
```

```
}

// 라우팅 설정

opts = append(opts, libp2p.Routing(func(h host.Host) (routing.PeerRouting, error

args := routingOptArgs

args.Host = h

r, err := params.RoutingOption(args)

out.Routing = r

return  r, err

}))

out.Host, err = params.HostOption(params.ID, params.Peerstore, opts...)

return  out, err

}
```

2. 피어 신원 관리

피어 ID 생성

IPFS 노드의 신원은 공개키에서 파생된 피어 ID로 식별됩니다:

```
func  PrivateKey(sk crypto.PrivKey) func(id peer.ID) (crypto.PrivKey, error) {

return  func(id peer.ID) (crypto.PrivKey, error) {

id2, err := peer.IDFromPrivateKey(sk)

if  err != nil {

return  nil, err

}
```

```
if id2 != id {
return nil, fmt.Errorf("private key in config does not match id: %s != %s", id,
}
return sk, nil
}
}
```

키 관리 과정

1. **키 생성**: 노드 초기화 시 Ed25519 또는 RSA 키 쌍 생성
2. **피어 ID 파생**: 공개키의 해시를 통해 고유한 피어 ID 생성
3. **키 저장**: 개인키는 안전하게 저장되고, 공개키는 피어스토어에 저장
4. **키 검증**: 연결 시 피어의 신원을 공개키로 검증

3. 연결 관리자 설정

IPFS는 연결 수를 효율적으로 관리하기 위해 연결 관리자를 사용합니다:

```
func ConnectionManager(low, high int, grace, silence time.Duration) func() (op
return func() (opts Libp2pOpts, err error) {
cm, err := connmgr.NewConnManager(low, high,
connmgr.WithGracePeriod(grace),
connmgr.WithSilencePeriod(silence),
)
if err != nil {
```

```
return opts, err

}

opts.Opts = append(opts.Opts, libp2p.ConnectionManager(cm))

return

}

}
```

연결 관리 정책

- **Low Watermark**: 최소 연결 수 (기본값: 600)
- **High Watermark**: 최대 연결 수 (기본값: 900)
- **Grace Period**: 새 연결의 보호 기간 (기본값: 10초)
- **Silence Period**: 연결 정리 간격 (기본값: 10초)

네트워크 전송 계층

지원되는 전송 프로토콜

IPFS는 다양한 전송 프로토콜을 지원합니다:

1. TCP 전송

```
if tcpEnabled {

opts.Opts = append(opts.Opts, libp2p.Transport(tcp.NewTCPTransport, tcp.WithMetr

}
```

2. QUIC 전송

```
if tptConfig.Network.QUIC.WithDefault(!privateNetworkEnabled) {
opts.Opts = append(opts.Opts, libp2p.Transport(quic.NewTransport))
}
```

3. WebSocket 전송

```
if wsEnabled {
opts.Opts = append(opts.Opts, libp2p.Transport(websocket.New))
}
```

4. WebRTC Direct

```
if tptConfig.Network.WebRTCDirect.WithDefault(!privateNetworkEnabled) {
opts.Opts = append(opts.Opts, libp2p.Transport(webrtc.New))
}
```

전송 계층 특징

| 프로토콜 | 특징 | 사용 사례 |

|---------|------|----------|

| TCP | 신뢰성 있는 연결 지향 | 일반적인 P2P 통신 |

| QUIC | UDP 기반, 빠른 핸드셰이크 | 저지연 통신 |

| WebSocket | 브라우저 호환성 | 웹 기반 노드 |

| WebRTC | NAT 트래버설 지원 | 방화벽 뒤의 노드 |

보안 계층 구성

지원되는 보안 프로토콜

IPFS는 두 가지 주요 보안 프로토콜을 지원합니다:

1. TLS 1.3

```
{
priority: tptConfig.Security.TLS,
defaultPriority: 100,
opt: libp2p.Security(tls.ID, tls.New),
}
```

2. Noise Protocol

```
{
priority: tptConfig.Security.Noise,
defaultPriority: 200,
opt: libp2p.Security(noise.ID, noise.New),
}
```

보안 협상 과정

1. **프로토콜 협상**: 연결 시 지원되는 보안 프로토콜 협상

2. **키 교환**: Diffie-Hellman 키 교환으로 세션 키 생성

3. **신원 확인**: 피어의 공개키로 신원 검증

4. **암호화 통신**: 협상된 암호화 알고리즘으로 데이터 보호

피어스토어 관리

피어 정보 저장

```
func Peerstore(lc fx.Lifecycle) (peerstore.Peerstore, error) {
pstore, err := pstoremem.NewPeerstore()
if err != nil {
return nil, err
}
lc.Append(fx.Hook{
OnStop: func(ctx context.Context) error {
return pstore.Close()
},
})
return pstore, nil
}
```

저장되는 정보

- **피어 ID**: 각 피어의 고유 식별자
- **공개키**: 신원 검증용 공개키

- **주소 정보**: 피어의 네트워크 주소 목록
- **프로토콜 정보**: 피어가 지원하는 프로토콜 목록
- **메타데이터**: 연결 품질, 지연시간 등

스트림 멀티플렉싱

멀티플렉싱 프로토콜

IPFS는 단일 연결에서 여러 스트림을 동시에 처리하기 위해 스트림 멀티플렉싱을 사용합니다:

- **yamux**: 기본 멀티플렉싱 프로토콜
- **mplex**: 대안 멀티플렉싱 프로토콜

스트림 관리

1. **스트림 생성**: 새로운 프로토콜 통신을 위한 스트림 생성
2. **스트림 식별**: 각 스트림에 고유 ID 할당
3. **플로우 컨트롤**: 스트림별 데이터 흐름 제어
4. **스트림 종료**: 통신 완료 후 스트림 정리

네트워크 주소 관리

주소 형식

IPFS는 multiaddr 형식을 사용하여 네트워크 주소를 표현합니다:

```
/ip4/192.168.1.100/tcp/4001/p2p/QmNodeID

/ip6/::1/tcp/4001/p2p/QmNodeID
```

```
/dns4/example.com/tcp/443/wss/p2p/QmNodeID
```

주소 발견 및 광고

1. **로컬 주소 발견**: 네트워크 인터페이스에서 사용 가능한 주소 탐지
2. **외부 주소 발견**: NAT 및 방화벽 뒤의 공개 주소 발견
3. **주소 광고**: DHT를 통해 다른 피어에게 주소 정보 전파
4. **주소 검증**: 연결 가능성 확인을 위한 주소 검증

설정 및 최적화

주요 설정 옵션

```
{
"Swarm": {
"ConnMgr": {
"LowWater": 600,
"HighWater": 900,
"GracePeriod": "10s"
},
"Transports": {
"Network": {
"TCP": true,
"QUIC": true,
"Websocket": true,
```

```
"WebRTCDirect": false
},
"Security": {
"TLS": 100,
"Noise": 200
}
}
}
}
```

성능 최적화 팁

1. **연결 수 조정**: 네트워크 환경에 맞는 연결 수 설정
2. **전송 프로토콜 선택**: 사용 사례에 적합한 전송 프로토콜 활성화
3. **보안 프로토콜 우선순위**: 성능과 보안의 균형을 위한 우선순위 설정
4. **대역폭 제한**: 네트워크 자원 관리를 위한 대역폭 제한 설정

문제 해결

일반적인 연결 문제

1. **NAT 트래버설 실패**: QUIC 또는 WebRTC 전송 활성화
2. **방화벽 차단**: WebSocket 또는 HTTPS 포트 사용
3. **피어 발견 실패**: 부트스트랩 노드 설정 확인
4. **연결 수 제한**: 연결 관리자 설정 조정

디버깅 도구

- **로그 레벨 설정**: `IPFS_LOGGING=debug`
- **연결 상태 확인**: `ipfs swarm peers`
- **주소 정보 확인**: `ipfs id`
- **네트워크 통계**: `ipfs stats bw`

이 문서는 IPFS에서 libp2p가 어떻게 통합되고 활용되는지에 대한 기본적인 이해를 제공합니다. 다음 문서에서는 피어 발견 메커니즘에 대해 더 자세히 다룰 예정입니다.